DOCUMENT

19

SOLID Design

How each principle is followed: the problem, the change and the proof

How the code follows the five SOLID principles: what was wrong, what changed, where to find it, and how a test proves it. The refactor changed structure only. Behaviour is identical: the 134 tests that existed before still passed unchanged, every case in `scripts/try_all_cases.py` still passes, and the mutation score is 100% (132 of 132).

1. At a glance

Principle	One line	Where it shows
Single responsibility	One reason to change per class	Four rule classes; `main.py`, `error_handling.py` and `auth/middleware.py` each do one job
Open/closed	Add behaviour by adding code, not editing code	A new approval rule is a new class listed in `standard_policy()`
Liskov substitution	Any implementation can replace another	The fake and the in-memory repository both pass one contract suite
Interface segregation	Depend only on what you use	`ApprovalService` takes an `ApprovalStore` (2 methods), not the 6-method repository
Dependency inversion	Depend on abstractions, injected from outside	The service receives a policy and a clock; `create_app` accepts replacements

2. Structure after the refactor

```
create_app(repository, users, policy)          <- the only place that picks concrete classes
   |
   +-- ApprovalService(store: ApprovalStore, policy: ApprovalDecisions, clock)
   |        |
   |        +-- ApprovalPolicy([ rule, rule, rule, rule ], limits)
   |                 ApproverRoleRule | ManagerLimitRule | DistinctApproversRule |
SecondApproverMustBeDirectorRule
   |
   +-- routers  -->  InvoiceReader / InvoiceWriter / ApprovalService / UserDirectory
   +-- error_handling.register_error_handlers(app)
   +-- auth.middleware.install_session_middleware(app)
```

3. Single responsibility

Problem. `api/main.py` built the app, mapped errors to status codes, wrote validation messages and checked sessions. Four reasons to change in one file. The approval rules were one function with four separate checks.

Change.

Before	After
`main.py`: wiring, error mapping, validation wording, session check	`main.py` wiring only; `error_handling.py`; `auth/middleware.py`
One function `rejection_reason` with four checks	`ApproverRoleRule`, `ManagerLimitRule`, `DistinctApproversRule`, `SecondApproverMustBeDirectorRule`
`ApprovalService` created its own policy dependency by importing the module	The service coordinates the lifecycle; the policy decides

Each rule class has one reason to change: the rule it states. `ApprovalPolicy` has one: how rules are combined.

Proof. `test_each_rule_refuses_only_its_own_case_on_its_own` and `test_each_rule_has_its_own_boundary_one_cent_either_side` test every rule in isolation.

4. Open/closed

Problem. To add a rule you had to edit the single rule function, risking the existing checks.

Change. A rule is anything with a `refusal(attempt)` method (the `ApprovalRule` Protocol). `ApprovalPolicy` asks each rule in order and reports the first objection. Adding a rule:

```
class BlockedUserRule:
    def __init__(self, blocked_username):
        self._blocked_username = blocked_username

    def refusal(self, attempt):
        if attempt.approver.username == self._blocked_username:
            return f"{self._blocked_username} is blocked from approving."
        return None

policy = ApprovalPolicy([ApproverRoleRule(), BlockedUserRule("manager.one")])
```

No existing rule and not `ApprovalPolicy` is touched. Limits are data (`ApprovalLimits(manager, director)`), so changing a limit is also not an edit to rule code.

Proof. `test_a_new_rule_plugs_into_the_policy_without_changing_the_existing_rules` defines `BlockedUserRule` inside the test file and uses it. `test_limits_are_injected_data_not_global_constants` runs the same rules with limits of 100 and 1,000.

5. Liskov substitution

Problem. The in-memory repository and the test fake were only assumed to behave alike. They did not: the real one gave a new invoice an id, the fake did not.

Change. `tests/test_repository_contract.py` runs the same 8 tests against both (16 results). The fake now assigns ids like the real repository. The contract covers unknown ids returning `None`, `add` and `get_by_id`, ids for invoices without one, `get_all` returning a snapshot, `get_by_ids`, `update` replacing, `update` of an unknown invoice raising `InvoiceNotFoundError`, and `remove` returning whether anything was removed.

Any future repository (for example a database) is added to the fixture and must pass the same tests. The rule classes follow the same idea: every rule has the same `refusal(attempt)` contract and `test_every_rule_has_no_objection_to_a_plain_director_approval_so_any_can_join_any_policy` checks they can all sit in any policy.

6. Interface segregation

Problem. `ApprovalService` received the full repository (six methods) but only ever called two.

Change. `domain/repository.py` now has small contracts:

Contract	Methods	Used by
`ApprovalStore`	`get_by_id`, `update`	`ApprovalService`
`InvoiceReader`	`get_all`, `get_by_id`, `get_by_ids`	list and get endpoints
`InvoiceWriter`	`add`, `update`, `remove`	create and delete endpoints
`InvoiceRepository`	all of the above	what an implementation offers

Proof. `test_the_service_works_with_a_store_that_offers_only_find_and_save` passes a class with only those two methods. If the service ever needed a third, that test would fail.

7. Dependency inversion

Problem. The service imported the concrete policy module and called the system clock directly. The login endpoint read a module-level list of demo users. None of these could be replaced without editing the code that used them.

Change.

Dependency	Abstraction	Provided by
Approval rules	`ApprovalDecisions` (`refusal_reason`, `required_approvals`)	`standard_policy()` by default; injected into `ApprovalService`
Time	`Callable[[], datetime]`	`utc_now` by default; injected
Storage	`ApprovalStore`	the repository passed to `create_app`
Users	`UserDirectory` (`authenticate`)	`DemoUserDirectory` by default; injected into `create_app`

`create_app(repository=None, users=None, policy=None)` is the composition root: the only place that chooses concrete classes. The approval count shown to the UI now comes from the same policy the service enforces (`ApprovalService.required_approvals`), so the two cannot disagree.

Proof. `test_the_service_obeys_whatever_policy_it_is_given_and_a_refusal_still_changes_nothing` (stub policy), `test_the_service_stamps_approvals_and_decisions_with_the_injected_clock` (fixed time), `test_login_uses_whichever_user_directory_the_app_was_built_with` (one-user directory), `test_the_approval_count_shown_to_the_ui_comes_from_the_policy_that_is_enforced` (different limits).

8. What deliberately did not change

- **Public functions** `rejection_reason`, `required_approvals` and `is_same_user` still exist as thin wrappers over the standard policy, so existing callers and tests keep working.
- **The lock** stays inside `ApprovalService`. Concurrency control is part of "do this lifecycle step safely", and moving it out would add machinery without removing a reason to change.
- **No framework or plug-in system.** Rules are listed in one function; there is no registry or auto-discovery. That would be more than this size of code needs.

9. Rules for future changes

1. A new approval rule is a **new class** plus one line in `standard_policy()`. Do not edit existing rule classes.
2. A new storage implementation must pass `tests/test_repository_contract.py`.
3. Code that needs storage declares the **smallest** contract it needs.
4. A new collaborator (clock, directory, policy) is **passed in**; only `create_app()` builds concrete ones.
5. If a file starts to have two reasons to change, split it.

These are also in `AGENTS.md` ("Design rules (SOLID)") so agents follow them.

10. Evidence

Check	Result
Tests before the refactor still passing unchanged	134 of 134
Tests now	177 (63 policy, 62 service, 36 API, 16 repository contract)
`python scripts/try_all_cases.py`	25 of 25
Mutation score	100% (132 of 132); first run left one survivor in a rule that had been hidden by an earlier rule, closed by per-rule boundary tests
Playwright test	1 passed